



## CREACIÓN DE CHATBOT SENCILLO

Se puede crear un chatbot sencillo con procesamiento del habla siguiendo un flujo bastante didáctico, usando herramientas gratuitas y fáciles de integrar en Google Colab o en entornos web.

### 1. Definir el objetivo y el dominio

- **Ejemplo:** Chatbot para responder preguntas frecuentes de una biblioteca escolar.

### 2. Elegir las herramientas

Para un prototipo rápido:

- **Reconocimiento de voz (voz → texto):**

Librería SpeechRecognition en Python o la API de Google Speech-to-Text.

- **Procesamiento de texto:**

Regla simple con if / elif o un modelo pequeño de NLP (como transformers de HuggingFace).

- **Síntesis de voz (texto → voz):**

Librería gTTS o pyttsx3.

- **Interfaz:**

Puede ser solo en consola o con Gradio para hacerlo interactivo.

### 3. Flujo básico del chatbot con PH

1. Escuchar la voz del usuario.
2. Convertir voz a texto.
3. Analizar el texto y detectar intención.
4. Generar respuesta.
5. Convertir la respuesta en voz.



## 4. Código de ChatBot sencillo (Versión A)

(versión muy sencilla para entender el ciclo completo)

### # ChatBot sencillo (Versión A)

#### # 1. Instalar librerías

```
!pip install SpeechRecognition gTTS gradio
```

```
import speech_recognition as sr
```

```
from gtts import gTTS
```

```
import gradio as gr
```

#### # 2. Función para procesar audio

```
def chatbot(audio):
```

```
    # Reconocimiento de voz
```

```
    r = sr.Recognizer()
```

```
    with sr.AudioFile(audio) as source:
```

```
        audio_data = r.record(source)
```

```
    try:
```

```
        texto = r.recognize_google(audio_data, language="es-ES")
```

```
    except:
```

```
        return "No entendí lo que dijiste", None
```

#### # Respuesta simple por reglas

```
if "hola" in texto.lower():
```

```
    respuesta = "¡Hola! ¿Cómo estás?"
```

```
elif "biblioteca" in texto.lower():
```

```
    respuesta = "La biblioteca abre de lunes a viernes de 8 a 18 horas."
```

```
else:
```

```
    respuesta = "No tengo esa información, pero puedo aprender."
```

#### # Texto a voz



```
tts = gTTS(respuesta, lang='es')
tts.save("respuesta.mp3")
return respuesta, "respuesta.mp3"
```

### # 3. Interfaz con Gradio

```
app = gr.Interface(
    fn=chatbot,
    inputs=gr.Audio(type="filepath"),
    outputs=[gr.Textbox(), gr.Audio()]
)
app.launch()
```

## Actividad 1 de ChatBot sencillo:

1. Grabar audios con posibles preguntas (Saludo inicial, solicitud de información de apertura de la biblioteca, Saludo final) que se ingresarán al entorno de Gradio del ChatBot.
2. Copiar el código del Chatbot sencillo (iterar para corregir la indentación).
3. Ejecutar e iterar esta **celda Versión A del ChatBot sencillo**. El resultado correcto permite la visualización de Gradio a la espera de la carga de los audios, luego presionar X para una nueva entrada de audio).

## 5. Código de ChatBot sencillo (Versión B)

### # ChatBot de ciclo continuo (Versión B)

```
import speech_recognition as sr
from gtts import gTTS
import gradio as gr
import os
```



### # Función para procesar una ronda de la conversación continua

```
def continuous_chatbot(audio_filepath, chat_history):  
    r = sr.Recognizer()  
    user_text = ""  
    bot_response_text = ""  
    bot_audio_filepath = "temp_response.mp3" # Archivo temporal para el audio del bot  
  
    try:  
        # Reconocimiento de voz del usuario  
        with sr.AudioFile(audio_filepath) as source:  
            audio_data = r.record(source)  
            user_text = r.recognize_google(audio_data, language="es-ES")  
            os.remove(audio_filepath) # Limpiar el archivo de audio del usuario  
  
        # Añadir el mensaje del usuario al historial para mostrarlo  
        # Usamos el nuevo formato de diccionario para gr.Chatbot type='messages'  
        chat_history.append({"role": "user", "content": user_text})  
  
        # Lógica de respuesta del chatbot  
        if "adiós" in user_text.lower() or "salir" in user_text.lower():  
            bot_response_text = "¡Hasta luego! Ha sido un placer conversar contigo."  
        elif "hola" in user_text.lower():  
            bot_response_text = "¡Hola! ¿Cómo estás? Dime, ¿en qué puedo ayudarte hoy?"  
        elif "biblioteca" in user_text.lower():  
            bot_response_text = "La biblioteca abre de lunes a viernes de 8 a 18 horas."  
        else:  
            bot_response_text = f"Dijiste: '{user_text}'. No tengo una respuesta específica para eso, pero puedo aprender."
```



```
except sr.UnknownValueError:
    bot_response_text = "No pude entender tu audio, por favor inténtalo de nuevo."
    user_text = "(Audio no reconocido)"
    chat_history.append({"role": "user", "content": user_text})
except sr.RequestError as e:
    bot_response_text = f"Error del servicio de reconocimiento de voz; {e}"
    user_text = "(Error de servicio)"
    chat_history.append({"role": "user", "content": user_text})
except Exception as e:
    bot_response_text = f"Ocurrió un error inesperado: {e}"
    user_text = "(Error inesperado)"
    chat_history.append({"role": "user", "content": user_text})
```

#### # Convertir la respuesta del bot a voz

```
if bot_response_text:
    try:
        tts = gTTS(bot_response_text, lang='es')
        tts.save(bot_audio_filepath)
    except Exception as e:
        print(f"Error generando TTS: {e}")
        bot_audio_filepath = None # No hay audio si falla el TTS
        bot_response_text += " (No pude generar audio de respuesta)"
```

#### # Añadir la respuesta del bot al historial

```
chat_history.append({"role": "chatbot", "content": bot_response_text})
```

```
return chat_history, bot_audio_filepath, chat_history
```

#### # Interfaz con Gradio para el ciclo continuo



```

with gr.Blocks() as app_b:
    gr.Markdown("### Chatbot Interactivo de Voz")
    gr.Markdown("Habla con el chatbot. El historial de la conversación se mostrará a continuación. Di 'adiós' o 'salir' para terminar.")
    # gr.State para mantener el historial de la conversación
    chat_history_state = gr.State([])

    # gr.Chatbot para mostrar el historial de la conversación
    # Actualizado con type='messages' y allow_tags=False
    chatbot_display = gr.Chatbot(height=300, label="Conversación", type='messages',
allow_tags=False)

    with gr.Row():
        # Entrada de audio del usuario
        user_audio_input = gr.Audio(label="Graba tu pregunta", type="filepath")
        # Salida de audio del bot
        bot_audio_output = gr.Audio(label="Respuesta del Chatbot", autoplay=True)

    # Conectar el evento 'stop' del audio de entrada a la función del chatbot
    user_audio_input.stop(
        continuous_chatbot,
        inputs=[user_audio_input, chat_history_state],
        outputs=[chatbot_display, bot_audio_output, chat_history_state]
    )
    # Iniciar la aplicación Gradio
    app_b.launch()

```

## Actividad 2 de ChatBot sencillo (Versión B):



1. Analizar la nueva celda de **Chatbot de Ciclo Continuo (Versión B)** que permite la continuidad del ingreso de audios a Gradio, hasta que el usuario decida terminar la consulta “ingresando el audio de saludo al ChatBot”.
2. Agregar al notebook la nueva celda de continuidad.
3. Ejecutar.
4. En celda de texto informar sobre **diferencias y agregados** realizados a la celda original para obtener la celda de continuidad, como también respecto resultados o sucesos obtenidos.

### Sugerencias:

- Se puede personalizar el conjunto de preguntas y respuestas.
- Pueden cambiar el dominio (turismo, salud, comercio local, etc.).
- Luego, en grupos, probar mejorar el reconocimiento o hacer respuestas más naturales.

### 6. Posible ampliación

- Usar un modelo de lenguaje de HuggingFace para interpretar intenciones.
- Guardar historial de conversación.
- Implementar reconocimiento de emociones en la voz.